Cloud Deployment Report

Data 790

Addison Orndorff

1. Introduction

After Milestone 1, a RAG pipeline was set up with a corpus of documents to give users insight into the NASA Apollo missions. After Milestone 2, that corpus was expanded, and security measures were put in place to harden the AI system. Now, that pipeline has turned into an application that has been deployed. Users should essentially be able to access the Apollo application publicly, submit questions, and get answers back that come straight from the corpus of NASA Apollo mission documents. In this cloud deployment project, the purpose was to deploy the application using Azure, Docker, and Kubernetes. Other things put in place were monitoring and observability with MLflow, auto-scaling, and a cost analysis.

2. Docker Containerization

To start out, I used the `rag_system` ZIP file given to us weeks ago right after Milestone 1. I added the `rag_service` folder to my Apollo RAG repository. Before running my application inside Docker, I had to prepare it first. I installed Azure CLI, signed in, ran it locally, and then I swapped in my Milestone 2 pipeline. I was able to build my Apollo RAG locally and test a question, to which I got a proper response. Then after running `./deploy.sh`, I built the Docker image and created the Azure container app. It took 2 minutes and 9 seconds to build, and Azure returned a public HTTPS address. A problem occurred, however, because the web address was not loading in my browser; the app startup kept failing.

I learned that this failure was because my app rebuilt the entire RAG database whenever a container would start; the startup was taking too long and failing. So, I added `persist_directory` to my `vector_store.py`. I prebuilt the Chroma database instead of rebuilding it each time the application would startup. I added a `chroma_db` folder, created `build_chroma_db.py`, and updated the Docker image to now have this prebuilt database. After deploying with Azure again, it succeeded and gave a public endpoint. However, while I was able to ask the applications questions and receive responses in the terminal, when I went to

the link, it said it was not found. I am not sure if I was supposed to see a working webpage corresponding to this link or not.

3. Kubernetes Configuration

Next was adding the Kubernetes configuration. I created a Kubernetes folder in my repository and created `deployment.yaml`, `service.yaml`, and `hpa.yaml`. The deployment file talks about how Kubernetes is intended to run with my Apollo RAG application; the service file discusses how other parts interact with the application; and the HPA file describes how Kubernetes can change the number of application replicas automatically, as HPA stands for horizontal pod autoscaler. Kubernetes is excellent for large scale cloud deployments, and manages containerized applications with auto-scaling.

4. Cloud Platform Deployment

The cloud provider I chose was Azure. I chose this because we had been using it already in this course, and in the Labs when working through the differences between Azure and AWS, it was recommended to stick with the provider you've already been using. Azure is nice because it has built-in content filtering, flexible quota measurements for cost management, and has lots of availability. A disadvantage with Azure however, that we discussed in our discussion posts for module 11, is that if you want to experiment with different models, you have to deploy them all.

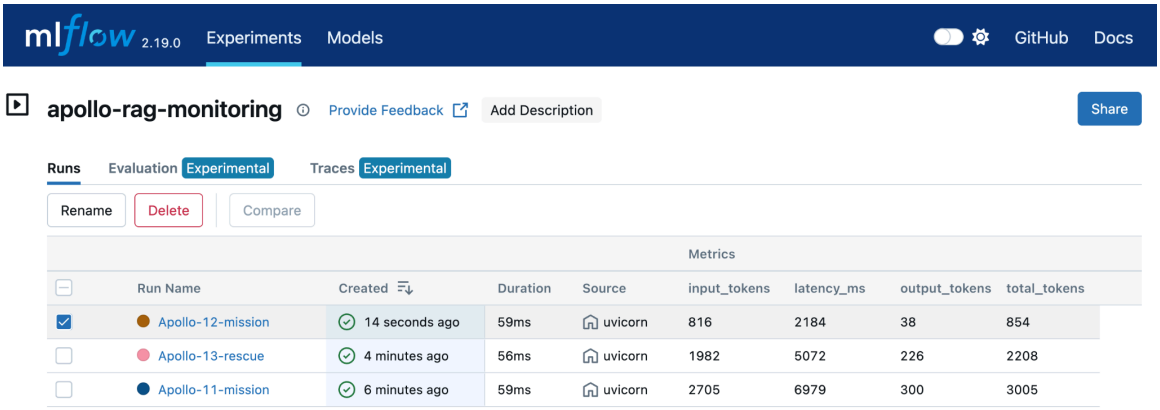
As far as the architecture of my deployment, Azure encapsulates three major things (as seen in my architecture diagram). First is the Azure container registry that stores my Docker image. It then deploys the image, where Azure Container Apps hosts the application. And lastly it runs the Docker container, which runs the Apollo RAG application.

5. Monitoring and Observability

It is very important to monitor our applications, because instead of just seeing the results, we can see what is really going on under the hood. By tracking and observing the parts of the application and the runs, if something goes wrong, we are able to look back at the experiments to analyze them. MLflow is an excellent platform that we can hook our models up to that monitors and observes them for us. It contains experiment tracking, a model registry, artifact logging, and visualizations.

In order to incorporate MLflow to my RAG system, I added token usage to three self-RAG calls in my `generation.py` file. I connected my system to MLflow and ran a few Apollo questions, which resulted in the runs appearing in the MLflow dashboard online. MLflow gathers latency metrics, input tokens, output tokens, total tokens, and if a request was blocked or not. The following screenshots show MLflow in action with my application. In the main dashboard view, we are able to see the experiment list of the RAG system. Here you can easily compare metrics side by side with one another. The three questions for each experiment are also included below. It's interesting that even though all three questions were about the Apollo missions, the Apollo 12 question asking for main accomplishments had a significantly lower total tokens amount of 854 tokens compared to the others which were 2208 and 3005 tokens. The latency of the Apollo 12 question was also less than half of each of the other questions. By clicking on a certain run, you can go in and see the metrics, and when you select multiple experiments, you are able to compare them in visualization form, like the graph produced below. In latency versus total tokens, all three runs are there, and you can see a trend going up; the more time an experiment takes to run, the more total tokens it uses.

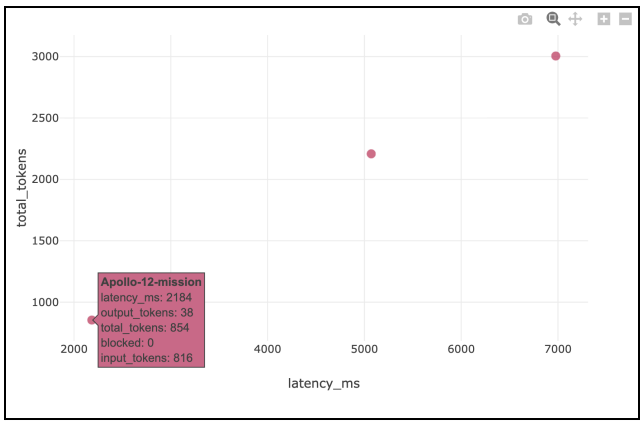
Experiment	**Question**
Apollo-12-mission	What were the main accomplishments of the Apollo 12 mission?
Apollo-13-rescue	What caused the oxygen tank explosion, and how did NASA bring the Apollo 13 crew home safely?
Apollo-11-mission	What were the major events leading up to the first moon landing?



Experiment list of Apollo RAG system

Metric	Value
latency_ms	6979
output_tokens	300
total_tokens	3005
blocked	0
input_tokens	2705

Apollo-11-mission question metrics



Run comparison visualization

6. Auto-scaling Configuration

I configured Azure container apps to have 0-2 replicas, which means Azure is able to scale my application between 0 and 2 running instances. The polling interval set in Azure is 30 seconds, which means this is the length of time in between the checks that Azure's scaling system makes, to check on my application and see if it needs more or less replicas. My application also has a 5 minute cool down period. This means that Azure will wait 5 minutes after requests stop coming in before shutting down to 0 running replicas. This gives the application a bit of a buffer; it thinks no requests are coming in, but just in case some come in during those 5 minutes, it still has a few things running. It does eventually shut off, however, and this helps reduce cost.

7. Cost Analysis and Optimization

It is important to analyze the costs of what my Apollo RAG application would generate. But where are those costs coming from? First, they're coming from Azure container apps, where my application is running. CPU and memory could create costs here by having replicas running. Then there's Azure Container Registry where the Docker image is being stored. This could potentially create costs, as this is where registry operations are going on. And last is the LLM's API, where questions are entered and answers are generated. These API calls each have input and output tokens that cost money.

To optimize spending, I configured my application to have a scale-to-zero feature, where if it's idle, it could have no running replicas. This is a great way to reduce computing costs, because no one is using the application, so it doesn't have to be running. My application has a maximum of 2 replicas. Because each replica has a limit of 0.5 CPU and 1 GB of memory, this maximum keeps the application from having an unlimited number of replicas, which keeps costs down from using too much CPU or memory. Another optimization strategy was having a prebuilt Chroma database. Before, the application was not working when I was rebuilding the database and generating embeddings each time a container would start. But now, it just loads the prebuilt database. This saves time and resources trying to repeatedly rebuild. And lastly, the RAG system has top k = 3, which means it gets three chunks per question asked. This limits how much document context is sent to the LLM, which cuts back on token costs.

8. Conclusion

The Apollo RAG system I have been building all semester is now deployed with its own API endpoint. It has been containerized with Docker in Azure and managed by Kubernetes. The Docker container runs the application. Through MLflow my RAG system can be monitored and observed. LLM-based metrics can be analyzed, like latency, input tokens, and output tokens. Auto-scaling measures have been put in place, like the 0-2 replicas scale range, and a 5 minute cool down period. The pre-built Chroma database was a way to fix the app, which was not starting up, and a way to reduce costs.

Though the Apollo RAG application is deployed and running, I do not think it's "production ready". I would want to go back and fix some things in the pipeline first. The main thing goes back to the very first milestone, when I had trouble retrieving quotes from the astronauts. This is important to me, because a big part of my corpus are onboard voice transcripts from the Apollo missions where the astronauts are just speaking; to one another, to mission control, to themselves, or to the cameras. In the future I hope to strengthen the retrieval ability of things they say, and I want the RAG to be able to identify which person said what.

Another future endeavor with this cloud deployment would be a front-facing webpage, where real users can find the URL and interact with the LLM by asking questions about the NASA Apollo missions from the 60-70s. It would be cool for users to ask questions about many different aspects of the missions, but also to discover exact quotes that the astronauts said. While large search engines like Google offer extensive information about these missions, I feel it would be difficult for users to access and examine real dialogue between the astronauts unless they access the Apollo RAG application.